

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	D. Yaswanth Sri Dana Ganesh	Reg. No.:	192425351
Date:		Duration:	60 minutes

Code of Conduct

I D.Yaswanth Sri Dana Ganesh (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Dana Ganesh

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

- 1. Problem Analysis**
 - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
- 2. Project Structure Design**
 - Design and explain the folder structure of your project repository.
 - Justify the purpose of each major folder and file included in the repository.
- 3. Git Repository Initialization**
 - Create a Git repository for the project.
 - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
- 4. Git Branching Strategy**
 - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
 - Justify why the selected branching model is appropriate for the assigned project.
- 5. Version Control Workflow**
 - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
 - Explain the purpose of each Git operation performed.
- 6. Commit History Analysis**
 - Present the commit history of the project.

- Explain how commit messages follow good version control practices and contribute to project maintenance.
- 7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
- 8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
- 9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
- 10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
- 11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
- 12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.

Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Analysis

1.1 Assigned Problem Statement

The assigned project is a Satellite Data Visualization Platform for Weather and Climate Pattern Analysis. Meteorological and climate science teams generate large volumes of satellite imagery every day — infrared, visible-light, and radar data — from agencies such as NASA and NOAA. This raw data is difficult to interpret without specialized scientific software, which excludes non-expert stakeholders such as policymakers, disaster management teams, agricultural planners, and students from participating in climate analysis. The project addresses this gap by building a web-based platform that ingests raw satellite data, processes it, and presents it through interactive maps and time-series charts that any user can understand without specialized training.

1.2 Core Problems Identified

- Satellite data is stored in complex scientific formats that require specialized software and expertise to interpret.
- Meteorologists spend significant manual effort processing raw imagery before patterns (cyclones, temperature shifts) become visible.
- There is no intuitive, web-based tool that lets non-technical users explore historical and real-time climate data interactively.
- Disaster management agencies often receive processed climate insight too late to act on it effectively.
- Cross-referencing multiple data types (temperature, precipitation, cloud cover) is a manual, fragmented process.
- Existing tools are expensive, proprietary, or too complex for students, small research teams, or local planning bodies.

1.3 Project Objectives

- Build a data ingestion pipeline for satellite imagery from public sources (NASA, NOAA).
- Design an interactive map-based module for real-time and historical weather visualization.
- Implement time-series analysis for temperature, rainfall, and cloud-cover trends.
- Build a pattern/anomaly detection feature (cyclone formation, droughts, temperature spikes).
- Provide a comparative visualization tool to overlay multiple data layers.
- Enable data export of processed datasets and visualizations.
- Provide region-specific filtering for localized analysis.
- Deliver all of this through a simple web interface requiring minimal technical skill.

1.4 Functional Requirements

- FR1: The system shall retrieve satellite imagery from public APIs on a scheduled basis.
- FR2: The system shall normalize and store processed data in a structured, queryable format.
- FR3: The system shall render satellite/climate layers on an interactive map.

- FR4: The system shall generate time-series charts for a user-selected region and date range.
- FR5: The system shall flag statistically significant anomalies automatically.
- FR6: The system shall allow authenticated users to export data as CSV, GeoTIFF, or image files.
- FR7: The system shall support region and date-range filtering across all visualizations.

1.5 Expected Outcomes

- Improved accessibility to satellite-based climate data for non-expert users.
- Faster identification of weather patterns and anomalies.
- Better decision-making support for disaster management during extreme weather events.
- Reduced manual processing time for researchers.
- A scalable foundation for adding more data sources and predictive analytics later.

2. Project Structure Design

2.1 Repository Folder Structure

```

satellite-viz-platform/
├── .git/                # Git metadata (auto-created)
├── .gitignore           # Files/folders excluded from version control
├── README.md            # Project overview, setup instructions
├── LICENSE              # Usage license
├── docker-compose.yml   # Multi-container orchestration
├── .env.example         # Sample environment variables
├── docs/
│   ├── architecture.md
│   └── api-spec.md
├── ingestion-service/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── src/
│       ├── fetch_nasa.py
│       ├── fetch_noaa.py
│       └── normalize.py
├── processing-service/
│   ├── Dockerfile
│   └── src/
│       ├── anomaly_detection.py
│       └── timeseries.py
├── backend-api/
│   ├── Dockerfile
│   ├── package.json
│   └── src/
│       ├── routes/
│       └── controllers/
├── frontend/
│   ├── Dockerfile
│   ├── package.json
│   └── src/
│       ├── components/
│       └── pages/
├── tests/
│   ├── unit/
│   └── integration/

```

2.2 Justification of Folder Structure

Folder / File	Purpose
.gitignore	Prevents build artifacts, node_modules, .env secrets, and large raw satellite files from being committed.
README.md	Onboards new contributors: setup steps, architecture summary, run instructions.
docker-compose.yml	Defines and wires together every microservice, network, and volume for local/prod deployment.
ingestion-service/	Isolated service responsible only for pulling and normalizing raw satellite data — kept separate so it can scale or fail independently of the rest of the app.
processing-service/	Runs anomaly detection and time-series computation; separated from ingestion so heavy compute doesn't block data collection.
backend-api/	Exposes processed data to the frontend through a versioned REST/GraphQL API.
frontend/	React-based UI: interactive maps, time-series charts, filters, export controls.
tests/	Unit and integration tests kept outside service folders so CI can run them uniformly across services.
docs/	Architecture notes and API specs, versioned alongside the code they describe.

3. Git Repository Initialization

3.1 Initialization Steps

```
$ mkdir satellite-viz-platform && cd satellite-viz-platform
$ git init
$ git config user.name "Your Name"
$ git config user.email "your.email@example.com"
$ echo "node_modules/\n __pycache__/\n.env\n*.log" > .gitignore
$ touch README.md LICENSE
$ git add .
$ git commit -m "chore: initial project scaffold"
$ git remote add origin https://github.com/<username>/satellite-viz-platform.git
$ git push -u origin main
```

3.2 Purpose of Essential Git Files

- .git/ — hidden directory holding the full object database, refs, and history; created automatically by git init and should never be edited by hand.
- .gitignore — tells Git which files/folders to never track (dependencies, secrets, cache, large raw data dumps), keeping the repository lean and secure.
- README.md — first file a collaborator reads; documents purpose, setup, and usage.
- LICENSE — defines the legal terms under which the code can be reused, important since the platform may reuse public satellite data and open-source libraries.
- .env.example — a template for required environment variables (API keys, DB credentials) without exposing actual secrets.

4. Git Branching Strategy

4.1 Selected Model: Gitflow

The project adopts the Gitflow branching model because the platform has clearly separated concerns (ingestion, processing, API, frontend) built by different contributors, a need for stable production releases (used by disaster-management stakeholders), and a recurring need for urgent fixes (e.g., a broken data feed during an active weather event) without disrupting ongoing feature work.

4.2 Branch Roles

- **main** — always reflects the latest stable, deployed version of the platform. Tagged with version numbers (v1.0.0, v1.0.1, ...).
- **develop** — integration branch where completed features are merged and tested together before a release is cut.
- **feature/*** — one branch per feature (e.g., feature/data-ingestion, feature/map-overlay), branched from develop and merged back via pull request.
- **release/*** — created from develop when preparing a new version; used for final QA, bug fixes, and documentation — merged into both main and develop on completion.
- **hotfix/*** — branched directly from main to fix urgent production issues (e.g., an ingestion service outage), then merged back into both main and develop.

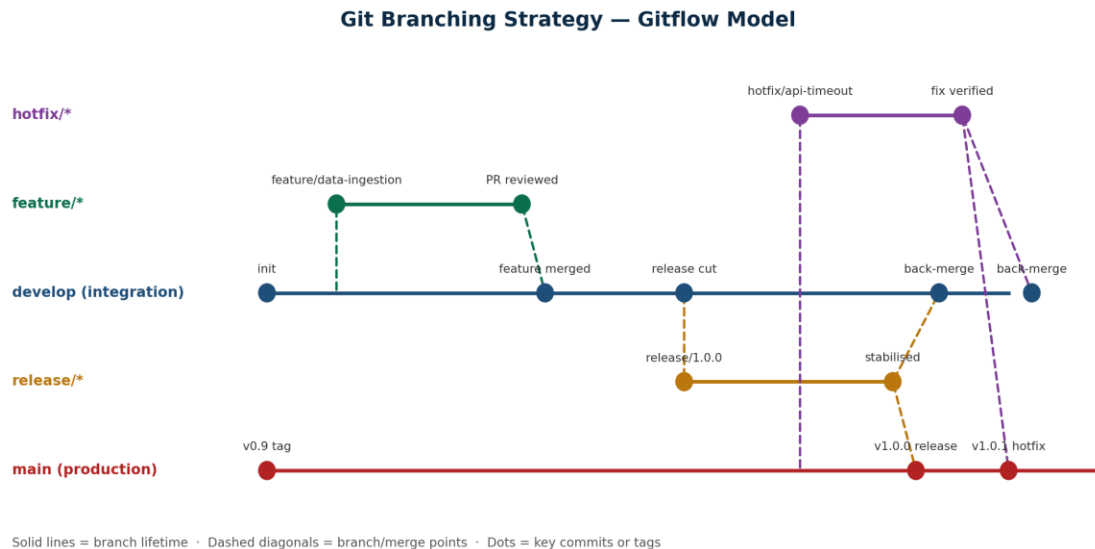


Figure 1: Git branching strategy (Gitflow) showing how feature, release, and hotfix branches interact with main and develop.

4.3 Why This Model Fits the Project

- Multiple independent components (ingestion, processing, API, frontend) can be developed in parallel feature branches without blocking each other.
- **develop** gives a safe integration point to catch conflicts before they reach production.
- Disaster-response stakeholders need **main** to always be deployable; release branches enforce a stabilization step before that happens.

5. Version Control Workflow

5.1 Example Feature Workflow

```
# create a feature branch from develop
$ git checkout develop
$ git pull origin develop
$ git checkout -b feature/data-ingestion

# work, then commit incrementally
$ git add ingestion-service/src/fetch_nasa.py
$ git commit -m "feat(ingestion): add NASA imagery fetcher"
$ git add ingestion-service/src/normalize.py
$ git commit -m "feat(ingestion): normalize raw imagery to GeoTIFF"

# push and open a pull request
$ git push -u origin feature/data-ingestion

# after review, merge into develop
$ git checkout develop
$ git merge --no-ff feature/data-ingestion
$ git push origin develop
$ git branch -d feature/data-ingestion
```

5.2 Conflict Resolution Example

A conflict can occur when two contributors edit the same section of docker-compose.yml (e.g., one adding the processing-service, another adding redis) on separate feature branches merged into develop close together.

```
$ git merge feature/redis-cache
Auto-merging docker-compose.yml
CONFLICT (content): Merge conflict in docker-compose.yml
Automatic merge failed; fix conflicts and then commit the result.

# open docker-compose.yml, resolve the <<<<<<< / ===== / >>>>>>> markers
$ git add docker-compose.yml
$ git commit -m "merge: resolve service definition conflict in docker-compose.yml"
```

5.3 Release and Synchronization

```
$ git checkout develop
$ git checkout -b release/1.0.0
# final QA fixes committed here
$ git checkout main
$ git merge --no-ff release/1.0.0
$ git tag -a v1.0.0 -m "First stable release"
$ git checkout develop
$ git merge --no-ff release/1.0.0
$ git push origin main develop --tags
```

5.4 Purpose of Each Operation

- `git checkout -b` — creates and switches to a new isolated line of work.
- `git add` / `git commit` — records a meaningful, reviewable snapshot of changes with an explanatory message.
- `git push -u` — publishes the branch to the shared remote and sets upstream tracking.

6. Commit History Analysis

6.1 Sample Commit Log

```
$ git log --oneline --graph --decorate
* 9f1a2c3 (HEAD -> main, tag: v1.0.0) merge: release 1.0.0 into main
|
|/
| * 7b3d4e5 (release/1.0.0) fix: correct timezone offset in time-series charts
| * 6c2b1a0 docs: update API spec for export endpoint
|/
* 5a9f8e7 (develop) feat(frontend): add layer-overlay comparison tool
* 4d8c7b6 feat(processing): add drought anomaly detection
* 3e7a6d5 feat(api): add region filtering endpoint
* 2f6b5c4 feat(ingestion): normalize imagery to GeoTIFF
* 1a5c4b3 feat(ingestion): add NASA imagery fetcher
* 0d4b3a2 chore: initial project scaffold
```

6.2 Commit Message Conventions Followed

- Conventional Commits format: <type>(<scope>): <description>, e.g. feat(ingestion): add NASA imagery fetcher.
- Types used: feat (new capability), fix (bug fix), docs (documentation only), chore (tooling/setup), merge (integration commits).
- Each message is written in the imperative mood ("add", not "added") and stays under ~72 characters for readability in git log.
- Commits are scoped to one logical change, which makes git bisect and code review far easier.

This discipline directly supports maintenance: a future contributor debugging a chart timezone bug can find fix: correct timezone offset in time-series charts in seconds, rather than searching through vague messages like "update" or "fix stuff".

7. Docker Environment Design

7.1 Why Docker Is Suitable

The platform is composed of several independently-scaling services with different runtimes (Python for ingestion/processing, Node.js for the API, React served via Nginx for the frontend, PostgreSQL and Redis as data stores). Docker packages each service with its exact dependencies, so it behaves identically on a developer's laptop, the CI pipeline, and production — removing “works on my machine” issues that are especially common with the geospatial libraries (GDAL, rasterio) this project depends on.

7.2 Components Requiring Containerization

Component	Why it's containerized	Base image
Ingestion service	Needs GDAL / geospatial libraries pinned to exact versions	python:3.11-slim
Processing service	Heavy numeric/analytics dependencies isolated from ingestion	python:3.11-slim
Backend API	Consistent Node runtime across environments	node:20-alpine
Frontend	Built React assets served by a lightweight web server	node:20-alpine → nginx:alpine

Component	Why it's containerized	Base image
Database	Reproducible PostGIS-enabled Postgres with persisted volume	postgis/postgis:16-3.4
Cache	Fast in-memory store for processed-tile caching	redis:7-alpine

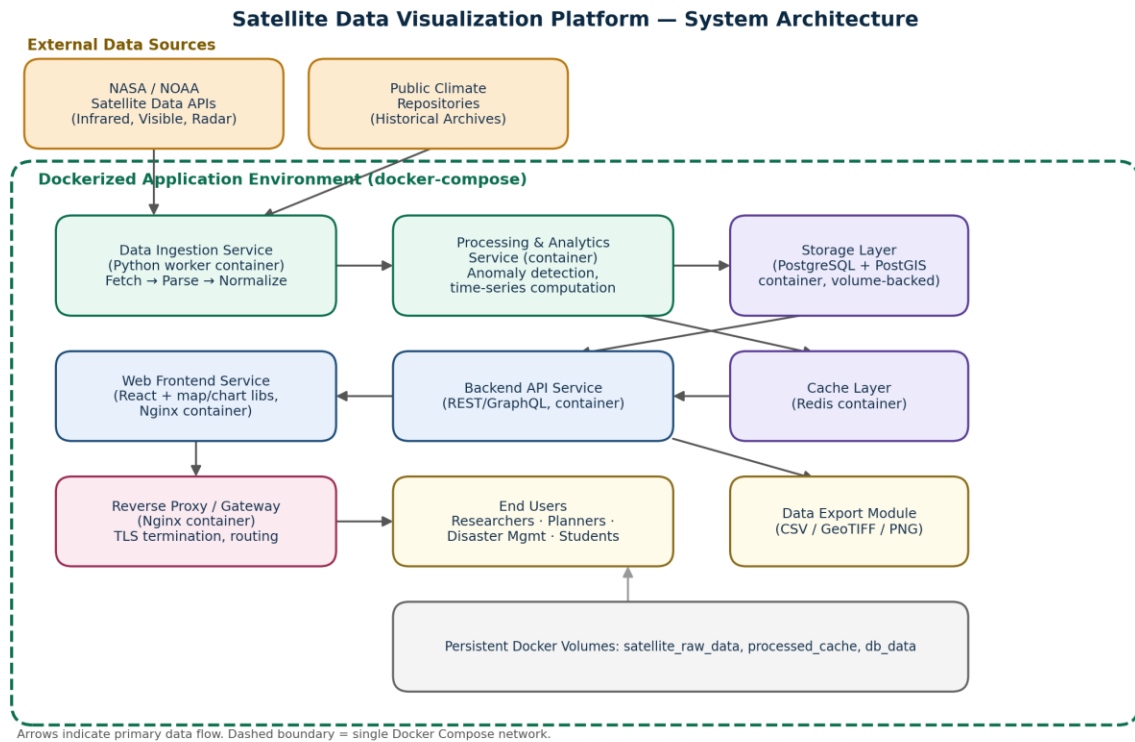


Figure 2: End-to-end system architecture, showing every containerized service inside the Docker Compose network.

8. Dockerfile Development

8.1 Dockerfile — Ingestion Service (example)

```

# ingestion-service/Dockerfile
FROM python:3.11-slim AS base

# System packages required by GDAL/rasterio for satellite imagery parsing
RUN apt-get update && apt-get install -y --no-install-recommends \
    gdal-bin libgdal-dev && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src/ ./src/

ENV PYTHONUNBUFFERED=1
USER 1000:1000

CMD ["python", "src/fetch_nasa.py"]

```

8.2 Instruction-by-Instruction Explanation

Instruction	Purpose
FROM	Selects a minimal, version-pinned base image to keep the final image small and reproducible.
RUN apt-get ...	Installs GDAL system libraries required to read satellite raster formats, then clears the apt cache to shrink image size.
WORKDIR	Sets a consistent working directory inside the container for all subsequent commands.
COPY requirements.txt	Copied before the rest of the source so Docker's layer cache reuses the installed dependencies unless requirements.txt changes.
RUN pip install	Installs Python dependencies inside the image, not on the host.
COPY src/	Copies application source last, so only this layer rebuilds on code changes.
ENV	Ensures Python logs stream immediately, which matters for real-time container log monitoring.
USER	Runs the process as a non-root user, reducing the security impact of a container compromise.
CMD	Defines the default command the container runs on start.

9. Docker Compose Design

9.1 docker-compose.yml

```
version: "3.9"

services:
  ingestion:
    build: ./ingestion-service
    env_file: .env
    volumes:
      - satellite_raw_data:/app/data
    depends_on:
      - postgres
    networks: [backend-net]

  processing:
    build: ./processing-service
    env_file: .env
    volumes:
      - satellite_raw_data:/app/data
      - processed_cache:/app/cache
    depends_on:
      - postgres
      - redis
    networks: [backend-net]

  backend-api:
    build: ./backend-api
    env_file: .env
    ports:
      - "4000:4000"
    depends_on:
      - postgres
      - redis
```

```

networks: [backend-net, frontend-net]

frontend:
  build: ./frontend
  ports:
    - "80:80"
  depends_on:
    - backend-api
  networks: [frontend-net]

postgres:
  image: postgis/postgis:16-3.4
  environment:
    POSTGRES_DB: satellite_db
    POSTGRES_USER: ${DB_USER}
    POSTGRES_PASSWORD: ${DB_PASSWORD}
  volumes:
    - db_data:/var/lib/postgresql/data
  networks: [backend-net]

redis:
  image: redis:7-alpine
  networks: [backend-net]

volumes:
  satellite_raw_data:
  processed_cache:
  db_data:

networks:
  backend-net:
  frontend-net:

```

9.2 Explanation of Key Elements

- Services — six services map one-to-one onto the architecture diagram: ingestion, processing, backend-api, frontend, postgres, redis.
- Networking — two isolated bridge networks: backend-net (data-layer traffic, never exposed to the internet) and frontend-net (only the API and frontend, which faces users). This limits the blast radius if the frontend is compromised.
- Volumes — satellite_raw_data and processed_cache persist large intermediate files across container restarts without bloating the images; db_data persists the PostGIS database so climate history survives redeployments.
- Environment variables — sourced from .env (never committed to Git) so credentials like DB_PASSWORD stay out of version control while remaining configurable per environment.
- depends_on — ensures dependent services (e.g., processing needs Postgres and Redis) start in a sensible order, though application-level readiness checks are still used for true health verification.

10. Container Deployment and Testing

10.1 Build and Run

```
$ docker compose build
$ docker compose up -d
$ docker compose ps
NAME                STATUS            PORTS
frontend            running          0.0.0.0:80->80/tcp
backend-api         running          0.0.0.0:4000->4000/tcp
postgres            running (healthy)
redis               running (healthy)
ingestion           running
processing           running
```

10.2 Testing Procedure

- Container health check: `docker compose ps` confirms every service reaches a `running` / `healthy` state.
- Service log check: `docker compose logs ingestion` confirms satellite data is fetched and normalized without errors.
- API smoke test: `curl http://localhost:4000/api/health` returns a 200 OK response.
- End-to-end test: opening `http://localhost` in a browser loads the interactive map and confirms a satellite layer renders.
- Integration test suite: automated tests under `tests/integration/` run against the composed stack in CI before merge.

```
$ curl -s http://localhost:4000/api/health
{"status":"ok","db":"connected","cache":"connected"}
```

Note: This section should be completed with your own terminal screenshots (`docker compose ps`, application screenshots, and API responses) captured from your running project, since evaluators require evidence from your specific build.

11. Benefits of Git and Docker

Benefit	How it applies to this project
Collaboration	Feature branches let the ingestion, processing, API, and frontend teams work in parallel without stepping on each other's code.
Version management	Tags (v1.0.0, v1.0.1) give disaster-management users a clear, citable version of the platform behind any given forecast.
Portability	Docker images run identically on a researcher's laptop, a university lab server, or a cloud VM.
Deployment speed	<code>docker compose up</code> brings the entire six-service stack online in one command instead of manual dependency installation.
Scalability	The processing service can be scaled independently (<code>docker compose up --scale processing=3</code>) during high satellite-data-volume periods, e.g. an active cyclone.
Maintainability	Clear commit history and isolated containers make it straightforward to trace, isolate, and fix issues without touching unrelated services.

12. Challenges and Improvements

12.1 Challenges Encountered

- Large satellite imagery files strain naive Git workflows; committing raw data directly bloats repository size.
- GDAL and geospatial dependencies increase Docker image size and build time compared to typical web services.
- Coordinating merges across five services increases the chance of docker-compose.yml conflicts when two features touch the same service definitions.
- Keeping environment variables consistent across ingestion, processing, and API containers requires careful .env management.

12.2 Suggested Improvements

- Adopt Git LFS (Large File Storage) for any sample satellite datasets that must be versioned.
- Use multi-stage Docker builds to strip build-time dependencies (compilers, headers) from the final runtime image.
- Introduce a CI pipeline (GitHub Actions) to run docker compose build and integration tests automatically on every pull request.
- Add a service mesh or API gateway as the platform grows, to centralize authentication and rate-limiting across services.
- Extend the pattern-detection module with a lightweight ML model for earlier anomaly warnings, as noted in the original objectives.